

Chapter2:Conceptual Data Modeling (ER Model)

Database System Concepts
University of Guilan

Yasaman Boreshban
Fall 2025

Outline

- Introduction to Data Modeling
- Entities, Attributes, and Relationships
- ER Diagrams (Symbols, Notations)
- Mapping Cardinalities (1:1, 1:N, N:M)
- Weak Entities and Identifying Relationships
- Enhanced ER Model (EER): Generalization, Specialization

Types of Data in a Database System

- One of the key elements of a database system is data.
- Data can be classified into two types:
 - User Data – data related to the application's real-world requirements.
 - System Data – metadata maintained by the DBMS itself.
- In this chapter, we focus on user data, the actual data that represents the real entities of the system.
- System data includes information about tables, columns, and relationships used internally by the DBMS.
- Our main concern here is to understand and model user data, which forms the foundation of logical database design

User vs System Data: Quick Examples

- User Data:

- Students, Courses, Enrollments, Grades
- Accounts & Transactions
- Products & Orders

- System Data (Metadata):

- Students(id, name, ...)
- Data types, primary/foreign key constraints
- Indexes on student_id
- Statistics for the optimizer
- User roles and privileges

- Derived Outputs:

- GPAs, summaries, dashboards
- Computed by queries/views, not stored unless required for performance.

Database Design Process

- Requirements Analysis
- Conceptual Design (ER Modeling)
- Logical Design (Relational Schema)
- Schema Refinement (Normalization)
- Physical Design & Implementation
- Database Usage & Maintenance

Data Modeling

- Data modeling involves a level of abstraction to represent only essential aspects of data.
- Not all real-world details are required in a database model.
- Meaningful and relevant user data are selected based on system requirements.
- Conceptual representation describes what data exists and how it relates to other data.
- The abstraction layer serves as the foundation for logical and physical design.

Languages for Data Modeling

- Data modeling requires a language to describe data and their relationships.
- The Entity–Relationship (ER) model is one of the most common approaches
- Other modeling approaches include:
 - UML (Unified Modeling Language)
 - Ontology-based modeling
- The ER model focuses:
 - real-world entities
 - Attributes
 - relationships

ER model -- Database Modeling

- The ER data model was developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database.
- The ER data model employs three basic concepts:
 - entity sets,
 - relationship sets,
 - attributes.
- The ER model also has an associated diagrammatic representation, the ER diagram, which can express the overall logical structure of a database graphically.

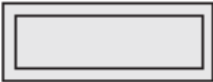
Entity Sets

- An entity is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- An entity set is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
 - Example:
 - instructor = (ID, name, salary)
 - course= (course_id, title, credits)

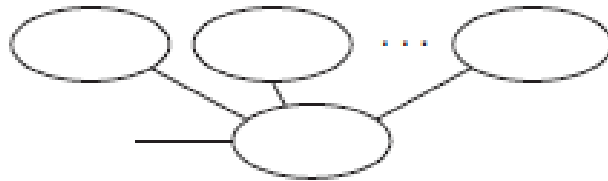
Identifying Entities, Relationships, and Attributes

- The first step in ER modeling is to identify **entities**, the key objects or concepts in the system.
- Each entity represents something that exists in the real world.
- The next step is to identify **relationships** among entities — how they are connected or interact.
- Relationships can have different **cardinalities** (1:1, 1:N, N:M).
- Each entity has **attributes** that describe its properties.
- Attributes may include:
 - **Identifier (key)** attributes that uniquely distinguish entities.
 - **Single-valued or multivalued** attributes.
 - More accurate and detailed information leads to more precise database design.
- Different **notations** exist for ER diagrams; this course will use a consistent simplified set of symbols.

ER Notations and Meanings

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute

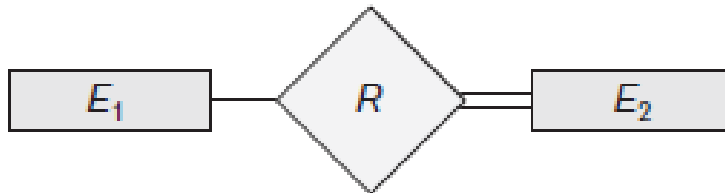
ER Notations and Meanings



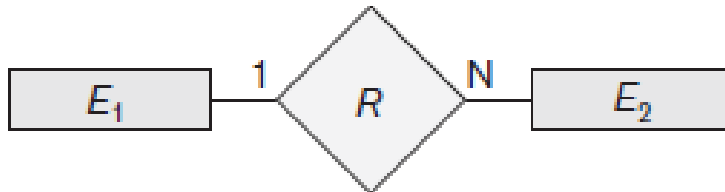
Composite Attribute



Derived Attribute



Total Participation of E_2 in R



Cardinality Ratio 1: N for $E_1:E_2$ in R

Entity Type – Definition and Characteristics

- An **entity type** represents any real-world object or concept about which data needs to be stored.
- In a **university environment**, examples include: *Student, Course, Instructor, Employee*.
- Each entity type should have the following characteristics:
 - A **distinct name** (e.g., Student, Instructor).
 - A **clear meaning or definition** within the context.
 - A **set of attributes** describing its properties (at least one).
 - At least **one instance** existing in the real world.
 - **Relationships** with other entity types
 - completely independent entities rarely exist.
- **Discussion point:** *Should “University” itself be considered an entity type?*

Conceptual Questions in Identifying Entities

- **Question 1:** Is there any entity type that has only **one instance** in the real world? If yes, should it still be modeled as an **entity** in the ER diagram?
 - *Example:* A university system may have only one “University” object.
 - .
- **Question 2:** If an entity has a **single attribute**, should it be modeled separately?
 - *Example:* An instructor has one publication (publisher). Should “Publication” be a separate entity, or just an attribute of “Instructor”?

Strong and Weak Entity Types

- **Strong Entity Type:**
 - An entity that exists **independently** in the real world.
 - Its existence does not depend on any other entity.
 - *Example: Course, Employee, Department.*
- **Weak Entity Type:**
 - An entity that **depends on another entity** (called the *owner* or *identifying entity*) for its existence. It cannot be identified without a related strong entity.
 - *Example: Section depends on Course; Dependent depends on Employee.*
 - A weak entity may depend on either a **strong** or another **weak entity**, forming an identifying relationship.
 - If the owner entity is deleted, the dependent entity usually **loses its meaning or existence**.

Clarifying the Concept of Weak Entity

- A **weak entity** is sometimes described as an entity **without its own identifier**, but this definition is **not always accurate**.
- The essential feature of a weak entity is **existence dependency**.
- It depends on another entity for identification and meaning.
 - Example: *Course Section*
 - Each section can be uniquely identified by a combination of *course_id, section_no*).
 - Although it has a **unique identifier**, it still **depends on the Course** for its existence.
- Therefore, a weak entity is not simply “identifier-less”
 - it is **existence-dependent** on another entity type.

Attributes – Definition and Constraints

- An **attribute** describes a **property or characteristic** of an entity type.
- If attributes are removed, the entity loses its **meaning and identity**.
- Each instance of an entity is identified by the **values of its attributes**.
- Each attribute has:
 - A **name**
 - A **specific meaning**
 - A **data type** (e.g., integer, string, date, etc.)

Attribute Constraints

- **Range constraint:** defines allowable values (e.g., 1–100).
- **Format constraint:** defines the presentation format (e.g., Date = YYYY-MM-DD).
- **Processing constraint:** defines logical rules (e.g., age always increases, never decreases).
- **Dependency constraint:** value of one attribute depends on another (e.g., foreign key).
- **Uniqueness constraint:** attribute values must be unique (e.g., identifier key).
- **Null constraint:** defines whether an attribute can be *null* or must have a value.

Identifier (Key Attribute) – Properties

- The **identifier** (or **key attribute**) uniquely distinguishes each entity instance.
- It must satisfy two essential properties:
 - **Uniqueness:** Each value identifies one and only one entity instance.
 - **Non-null:** The key attribute must always have a value (cannot be null).
 - *Example:* A student's ID number cannot be empty.
- Two additional desirable properties (not mandatory):
 - **Short and simple:** Shorter keys improve readability and efficiency.
 - **Stable over time:** Key values should not change frequently to avoid unnecessary updates.

Simple, Composite, and Derived Attributes

- **Simple Attribute:**
 - Has indivisible values that cannot be broken down further.
 - *Example: FirstName, NationalID, Salary.*
- **Composite Attribute:**
 - Can be divided into smaller sub-parts, each representing a meaningful component.
 - *Example: Address → (Street, City, PostalCode).*
- **Derived Attribute:**
 - Value can be **calculated or derived** from other stored attributes.
 - Usually **not stored** in the database.
 - *Example: Age can be derived from DateOfBirth.*
- **Discussion Point:** Should *Address* **always** be modeled as a composite attribute?

Attribute Classifications

- Attributes can be classified in several ways:
- **By Identification Role:**
 - **Key Attribute:** Has unique values used to identify entity instances.
 - **Non-Key Attribute:** Any other attribute not used for identification.
- **By Structure:**
 - **Simple Attribute:** Cannot be divided into subparts.
 - **Composite Attribute:** Can be divided into smaller attributes (e.g., *Address* → *Street*, *City*, *ZipCode*).
- **By Number of Values:**
 - **Single-Valued:** One value per entity instance (e.g., *BirthDate*).
 - **Multivalued:** Multiple values per entity instance (e.g., *Phone Numbers*).
- **By Nullability:**
 - **Nullable:** May have no value (*NULL* allowed).
 - **Non-nullable:** Must always have a value (*NOT NULL*).

The Concept of NULL

- **NULL** represents the **absence of a known value**, not zero or an empty string.
- Two common meanings of NULL:
 - The value **exists**, but is **currently unknown**.
 - Example: A student's *phone number* not yet provided.
 - The value **does not apply** or is **undefined** for that entity.
 - Example: *Spouse Name* for an unmarried student.
- Whether an attribute is **nullable or not** must be defined in the **DBMS schema**.
- Managing NULLs can be **challenging** in database operations and integrity checks.

Stored (Real) vs. Derived (Virtual) Attributes

- **Stored (Real) Attribute:**
 - The value is **physically stored** in the database.
 - Represents actual data collected and maintained.
 - *Example: DateOfBirth, Salary*
- **Derived (Virtual) Attribute:**
 - The value is **computed** from other stored attributes.
 - Not usually stored in the database.
 - *Example: Age derived from DateOfBirth.*
- **Discussion Point:**
 - Are derived attributes **never stored**?
 - sometimes they are stored for **performance** or **historical** reasons, e.g.:
 - Pre-computed *TotalPrice* in orders table.
 - Cached *GPA* for reporting efficiency.

Relationship Type – Definition and Characteristics

- A **relationship type** represents an **interaction or association** among two or more entity types.
 - *Example: Student – Enrolls – Course*
- **Binary relationship:** involves two entity types.
- **Ternary relationship:** involves three entity types and is **not equivalent** to multiple binary ones.
- **Recursive (self-referencing)** relationship: an entity type participates with itself.

Relationship Type – Characteristics

- **Main Characteristics:**

- Has a **name** (e.g., *Enrolls*, *Teaches*, *Supervises*)
- Has a **clear meaning** within the domain
- Has **participating entity types**

- **Participation Constraint:**

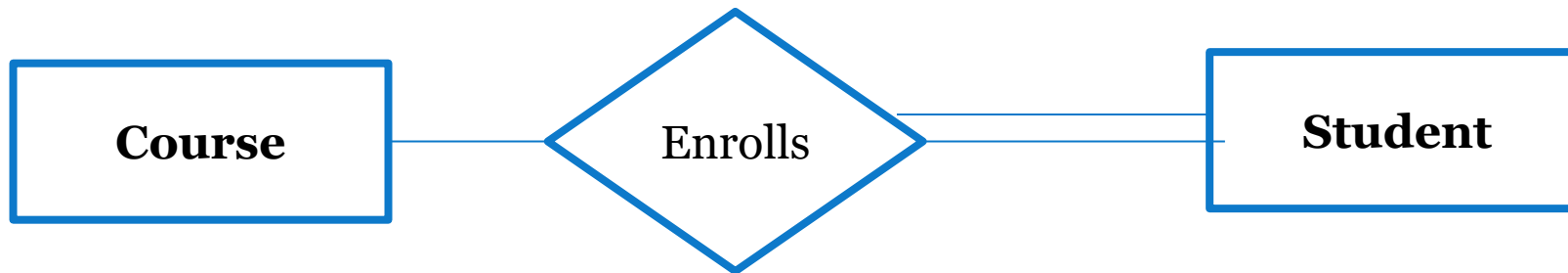
- **Total participation:** every instance of the entity **must** participate in the relationship.
- **Partial participation:** some instances **may not** participate in the relationship.

Examples of Participation Constraints

- **Example 1 – Instructor and CourseGroup**

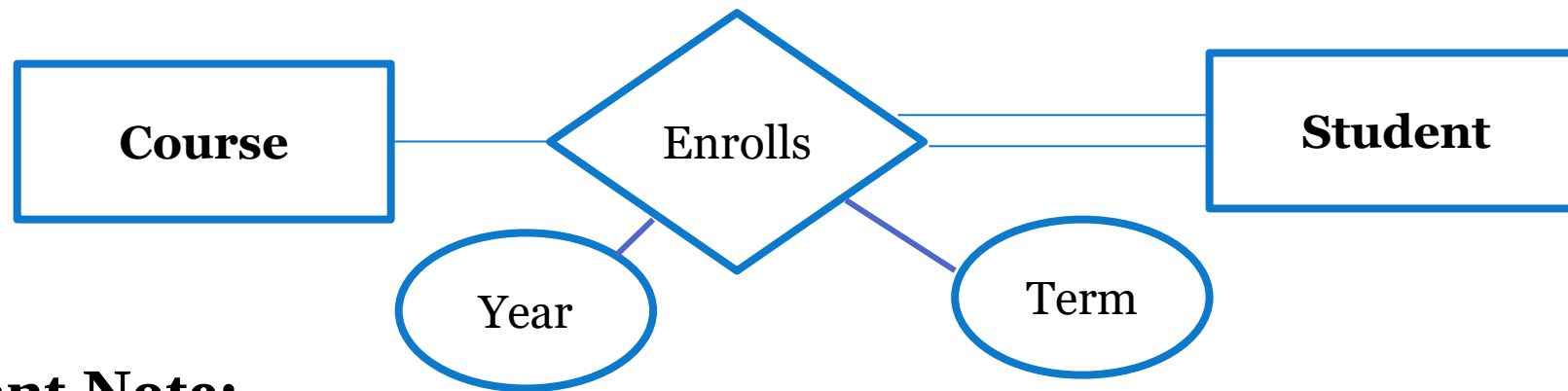


-
- **Example 2 – Student and Course**



Relationship Type -Attributes

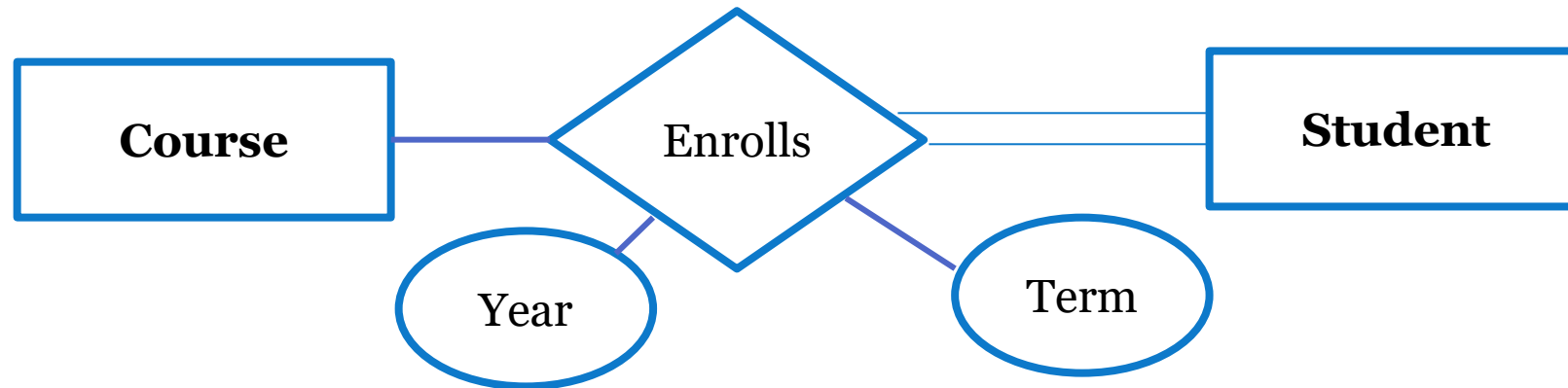
- A **relationship type** may have its own **attributes** describing the interaction between entities.
- *Example:* In *Student–Enrolls–Course*, attributes like *Year* and *Term* describe **when** the enrollment occurred.



Important Note:

- Each **relationship instance** is **uniquely identified** by the participating entity instances involved in it. (*Silberschatz et al., Database System Concepts*)

Discussion: What Is the Problem Here



- Can a student enroll in the **same course** in two different terms?
- How would this be represented in the ER model?

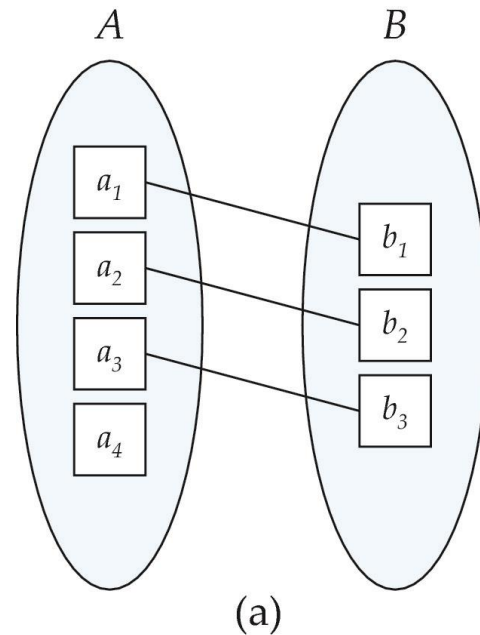
Solutions to the Enrollment Relationship Problem

- **Solution 1 – Multivalued Attribute Approach:**
 - Treat *Term* and *Year* as **multivalued attributes** of the relationship.
 - Later, during logical design, create a separate table including:
(StudentID, CourseID, Term, Year)
- **Solution 2 – Separate Entity Approach:**
- Model *Term* and *Year* as part of a new entity type

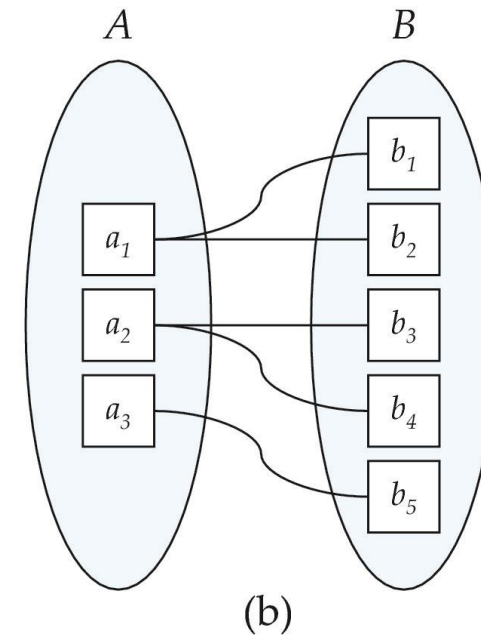
Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many

Mapping Cardinalities



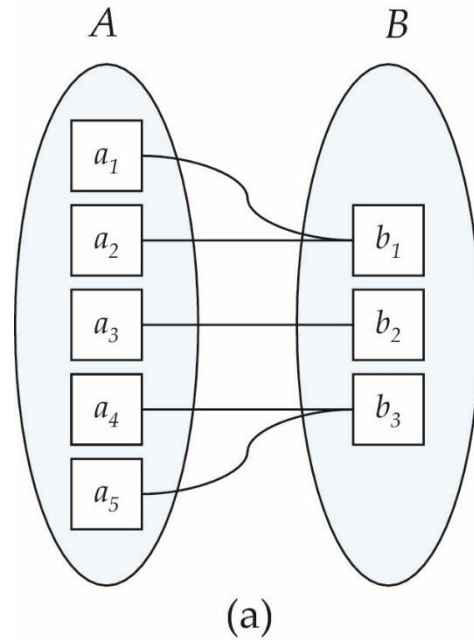
One to one



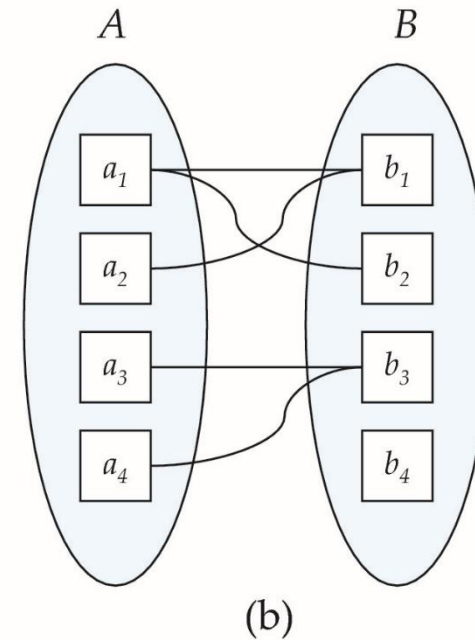
One to many

- Note: Some elements in A and B may not be mapped to any elements in the other set

Mapping Cardinalities



Many to one



Many to many

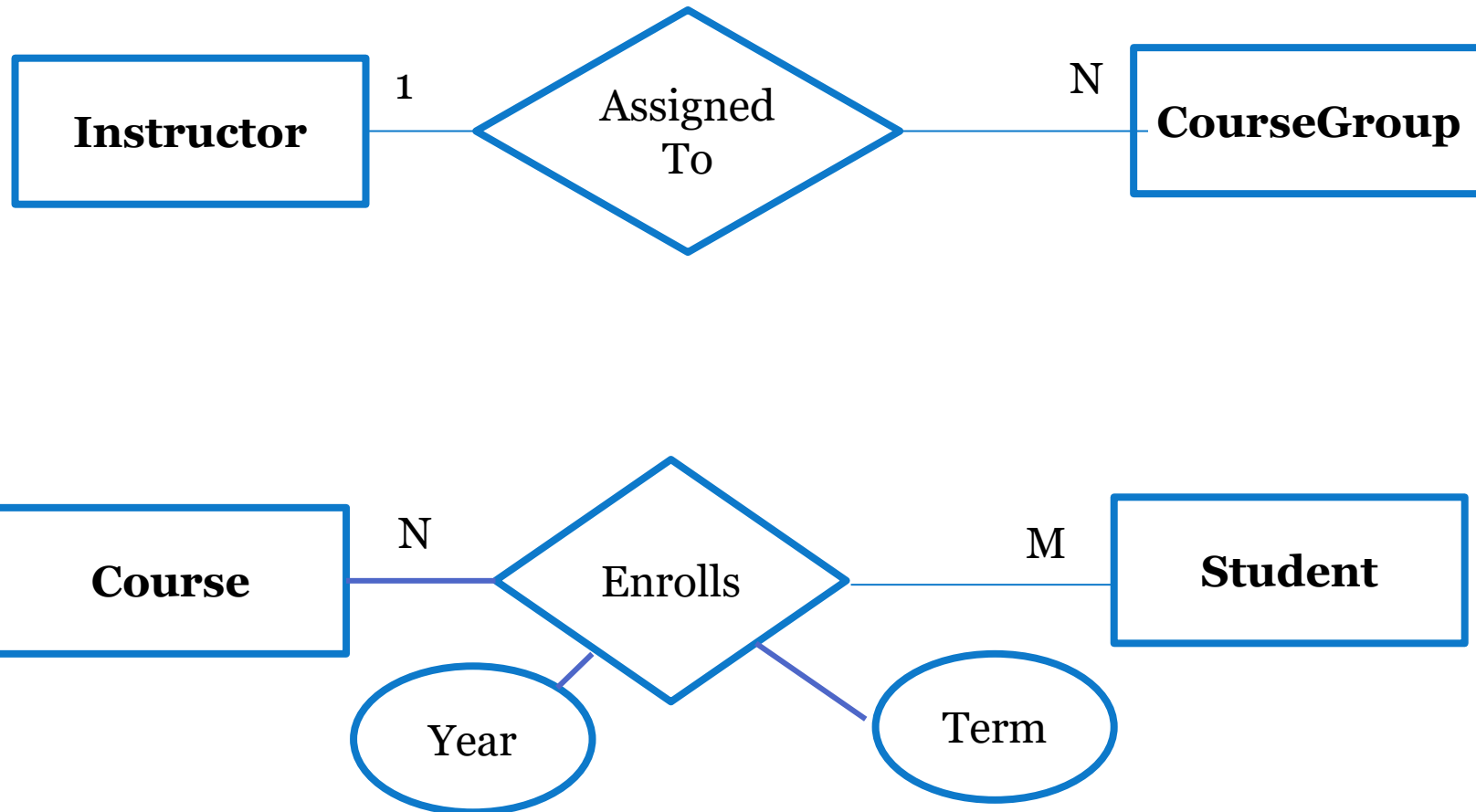
- Note: Some elements in A and B may not be mapped to any elements in the other set

Mapping Cardinalities in Binary Relationships

- Mapping cardinality defines **how many instances** of one entity can be associated with **instances of another entity** in a binary relationship.
- Possible types:
- **One to One (1:1)**: Each entity instance in A relates to at most one in B, and vice versa.
 - *Example*: Each person has one passport.
- **One to Many (1:N)**: Each entity in A can be related to many in B.
 - *Example*: One instructor teaches many courses.
- **Many to One (N:1)**: Many entities in A relate to one in B.
 - *Example*: Many students belong to one department.
- **Many to Many (M:N)**: Entities from both sides can relate to many from the other.
 - *Example*: Students enroll in many courses, and each course has many students.

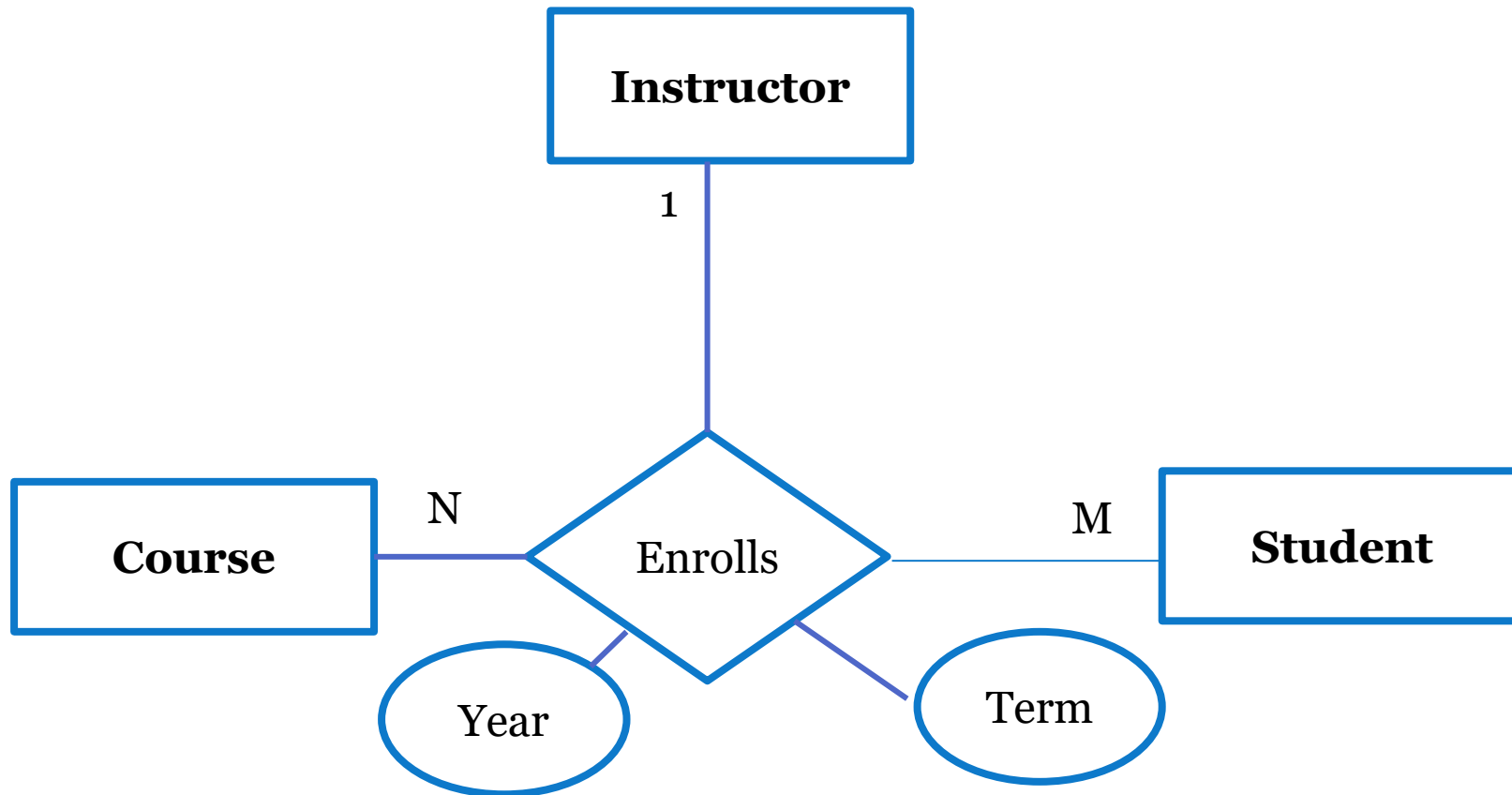
Example – Relationships and Cardinalities

- To determine cardinality, analyze both directions



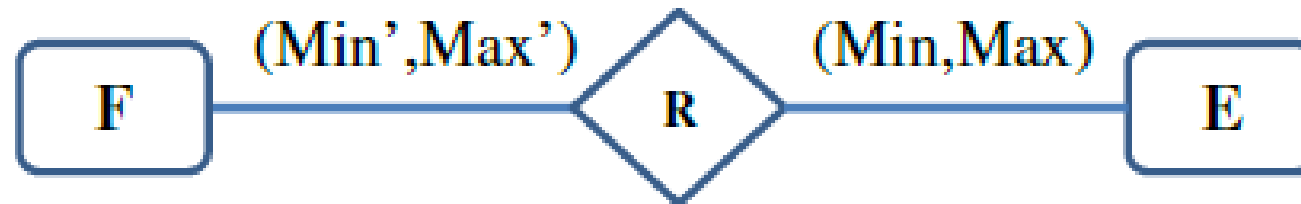
Example – Ternary Relationship

- Fix two entities at a time and analyze the third



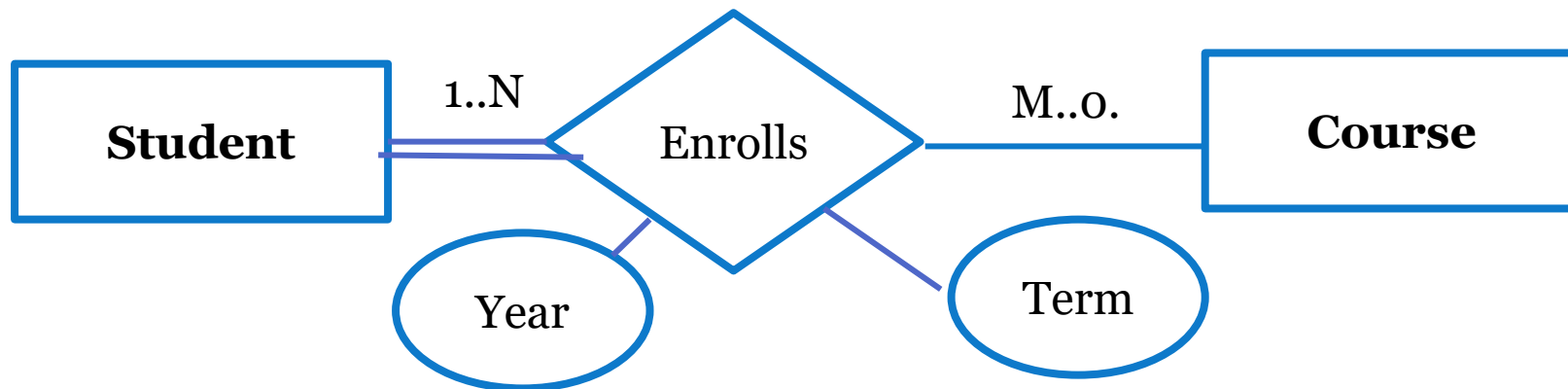
Alternative Notation for Cardinality – (Min, Max) Form

- Another way to represent participation and cardinality constraints is the (Min, Max) notation.
- For each entity instance **e** of type **E**,
 - It must participate in **at least Min** and **at most Max** instances of relationship **R**.
 - This notation shows **both the lower and upper bounds** of participation



Example – (Min, Max) Notation in ER Diagrams

- **Example:**
- A student can enroll in *between 1 and N courses*,
- and a course may be taken by *between 0 and M students*.

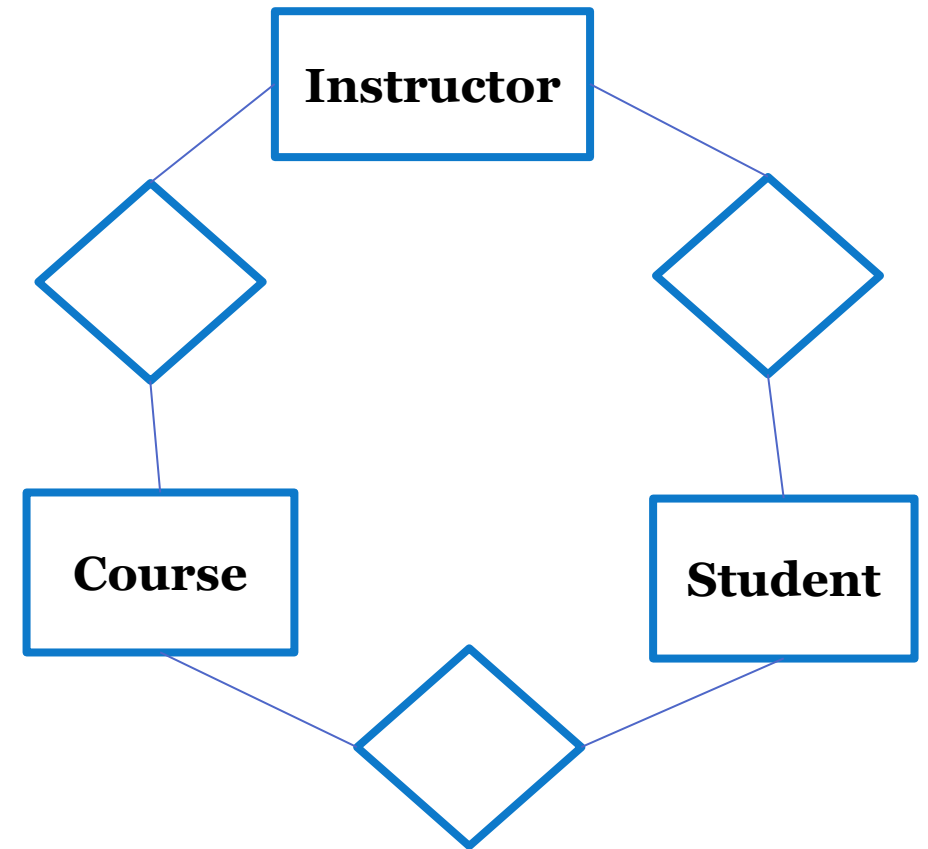
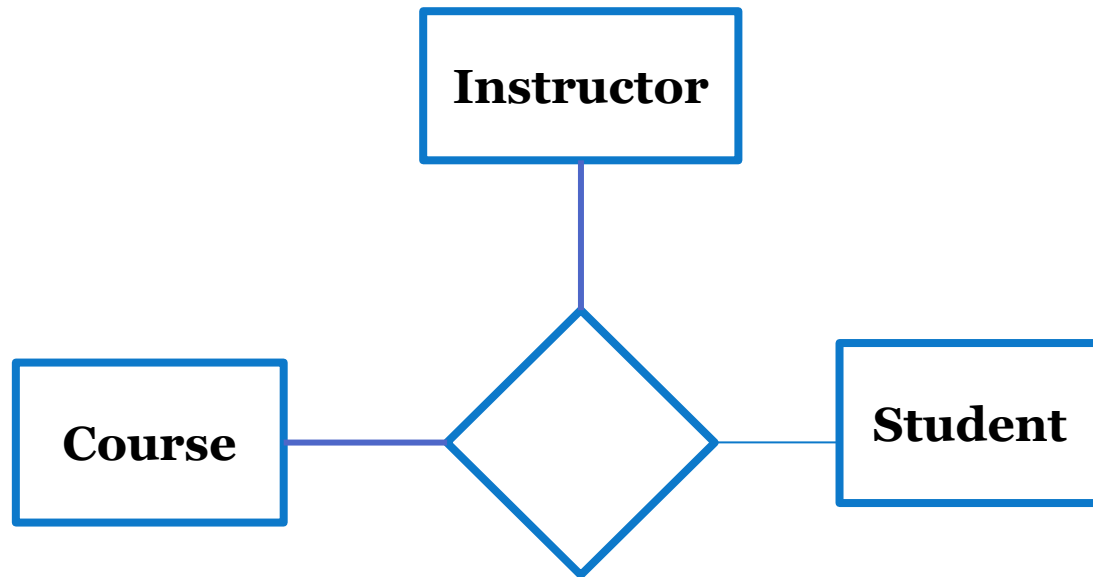


- **Question:**
- What are the **advantages** of using (Min, Max) notation?

Binary vs. Ternary Relationships in ER Modeling

- Sometimes a relationship seems to involve three entities.
- We might represent it either as:
 - **Three binary relationships**, or
 - **One ternary relationship**.
- But these two representations are **not equivalent** in meaning.
- **Question for discussion:**
- Can we always replace a ternary relationship with binary ones?

Example – Binary vs. Ternary Relationships



- **Important:** A ternary relationship and a set of binary relationships are not equivalent. Some information may exist in one and not in the other.

Definition of Weak Entity

- A weak entity is an entity type that has existence dependency on another entity. If the owner entity is deleted, the weak entity must also be deleted.



- The distinction between strong and weak entities is relative.
- An entity may be weak with respect to one entity, but strong with respect to another.
- A weak entity has no complete identifier of its own
 - include a discriminator (**partial key**) to distinguish its instances.
 - The discriminator is unique only among the weak entities related to the same owner, but not globally unique

Key Characteristics of Weak Entities

1. Existence dependency:

- A weak entity cannot exist without its owning (strong) entity.

2. No independent key:

- It does not have a complete identifier of its own; uniqueness is achieved **by combining** its partial key with the key of the owner entity.

3. Total participation:

- The identifying relationship must be **mandatory** from the weak entity side.

4. Relationship type:

- The identifying relationship is always **1–N** or possibly **1-1**

5. Question:

- *Can a weak entity have an N-to-N relationship with its owner?*

When to Use Weak Entities

- Weak entities are useful in several common modeling situations:

1. Representing multivalued or composite attributes:

- A multivalued attribute can be modeled as a weak entity.
- *Example:* Educational degrees of an employee (*Employee–Has–Degree*).

2. Modeling repeating or time-dependent events:

- Repeated occurrences that depend on another entity can be modeled as weak entities.
- *Examples:* Rewards, disciplinary actions, assignments, or mission records related to an employee.

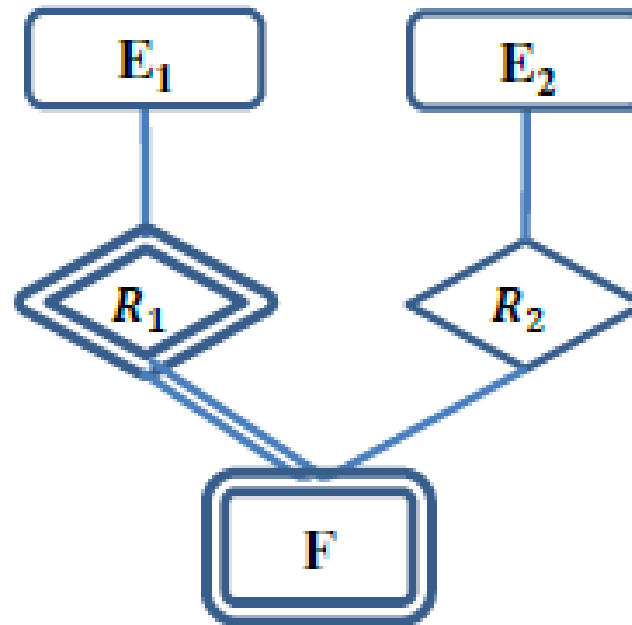
3. Converting ternary relationships into binary form:

- Sometimes a ternary relationship can be replaced with a weak entity that connects two binary relationships.
- *Example:* The relationship among *Student–Course–Instructor* can be represented as a weak entity (e.g., **Enrollment**).

Note – Weak Entity Can Have Other Relationships

- A **weak entity** is not restricted only to its **identifying relationship**.
- It can also participate in **other strong (non-identifying) relationships**.
- The identifying relationship defines **existence dependency**, while other relationships represent **additional associations**.

- **Example:**
 - CourseGroup



Extended ER Model (EER)

- “IS-A” Relationship:

- *Specialization: creating subtypes from a general entity.*
- *Generalization: forming a supertype from several entities with common attributes.*

- “PART-OF” Relationship:

- *Decomposition: breaking an entity into components.*
- *Composition: combining several entities into a whole.*

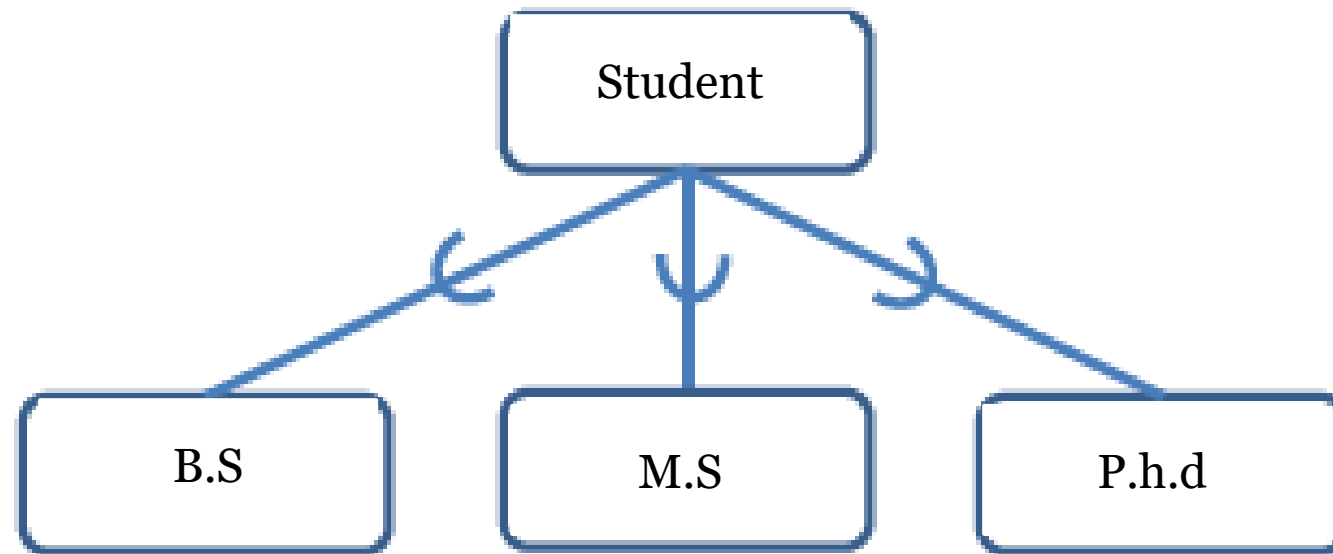
- Aggregation:

- *Relationship on Relationships*

“IS-A” Relationship

- The IS-A relationship represents a link between a **supertype** and one or more **subtypes**.
- It defines a hierarchical structure where subtypes inherit attributes and relationships from the supertype.
- The distinction between subtypes is usually based on a defining attribute.
- The notation is written as:
 - *Subtype IS-A Supertype*
- Two modeling techniques:
 - *Specialization: deriving specific subtypes from a general entity.*
 - *Generalization: forming a general entity from multiple specific entities.*

Example: “IS-A” Relationship



Types of Specialization: Complete vs. Partial

- A specialization can be **complete** or **partial**, depending on whether all instances of the supertype must belong to a subtype.

1. Complete Specialization:

- Every instance of the supertype **must appear** in at least one subtype.
- Represented by a **double line** connecting the supertype to the specialization.

2. Partial Specialization:

- Some instances of the supertype **may not belong** to any subtype.
- Represented by a **single line** connecting the supertype to the specialization.

Types of Specialization: Disjoint vs. Overlapping

- A specialization can be **disjoint** or **overlapping**, depending on whether an instance of the supertype can belong to more than one subtype.

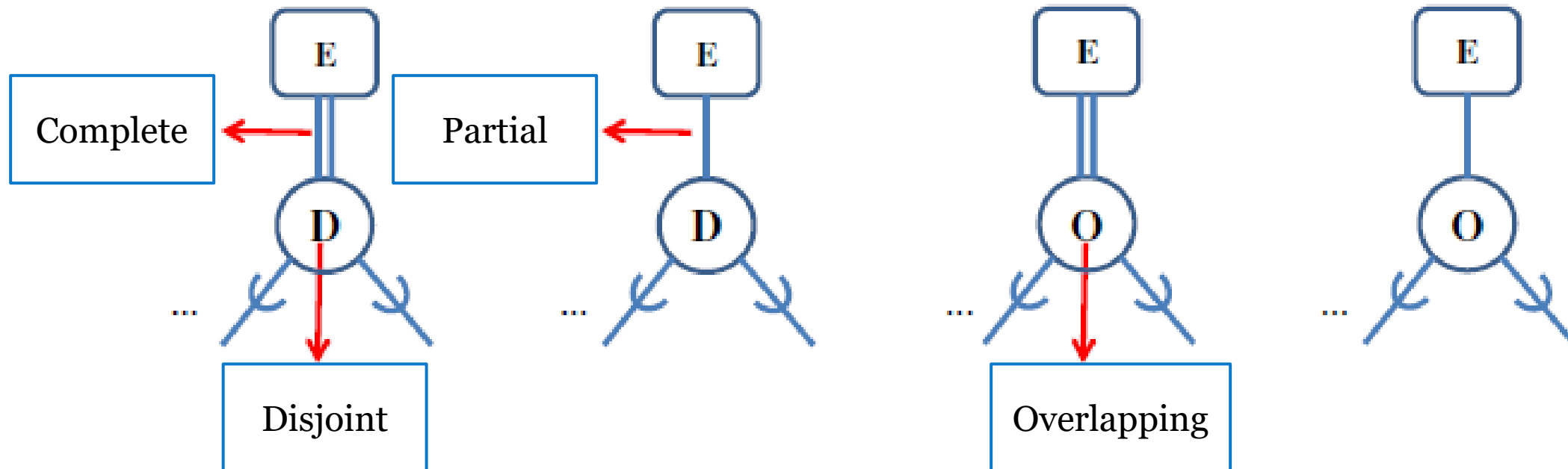
1. Disjoint Specialization:

- Each instance of the supertype **belongs to at most one** of the subtypes.
- Represented by a “**D**” in the circle connecting the subtypes.

2. Overlapping Specialization:

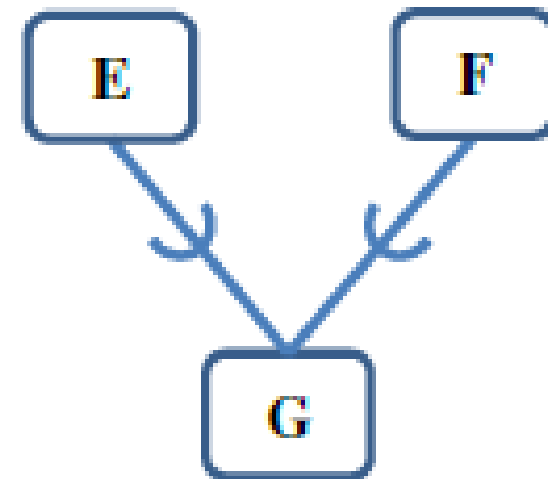
1. Each instance of the supertype **may belong to more than one** subtype.
2. Represented by an “**o**” in the circle.

Types of Specialization – Combined Constraints



Multiple Inheritance in Specialization

- Multiple inheritance occurs when a **subtype** is a specialization of **more than one supertype**.
- This means that the subtype inherits attributes and relationships from **all** of its supertypes.
- It is useful when an entity naturally belongs to multiple higher-level categories.
- **Question:** What should be the
- **identifier (key) of a subtype that inherits**
- **from multiple supertypes?**



Generalization

- Generalization is the process of identifying a new, higher-level entity type (supertype) from two or more existing entity types that share common features.
- It is the reverse of specialization.
- Typically used when several entity types have similar attributes or relationships.
- reducing redundancy by grouping common properties into one generalized entity.

Conditions for Generalization

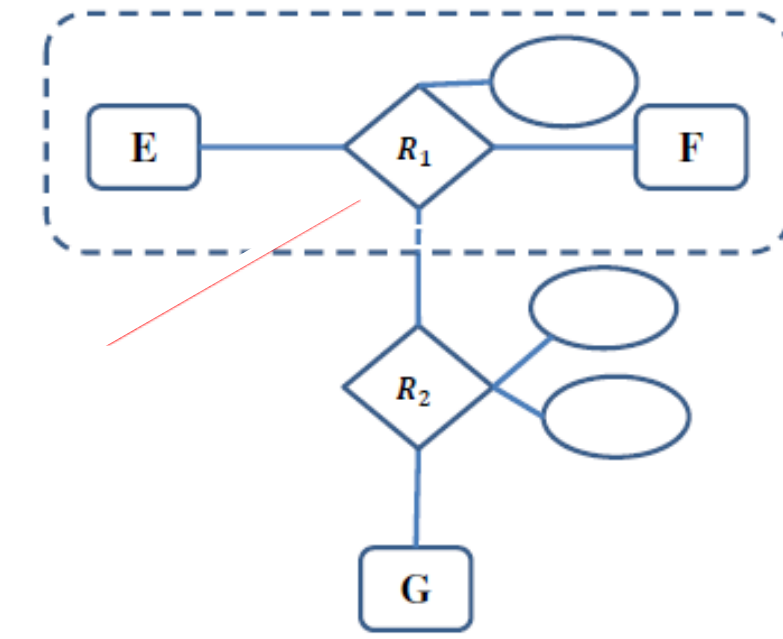
1. The entity types should have a common identifier
 - their keys come from the same domain
2. There must be at least two subtypes involved.
3. The more common attributes the entity types share

“IS-A-PART-OF” or “Has-A” Relationship

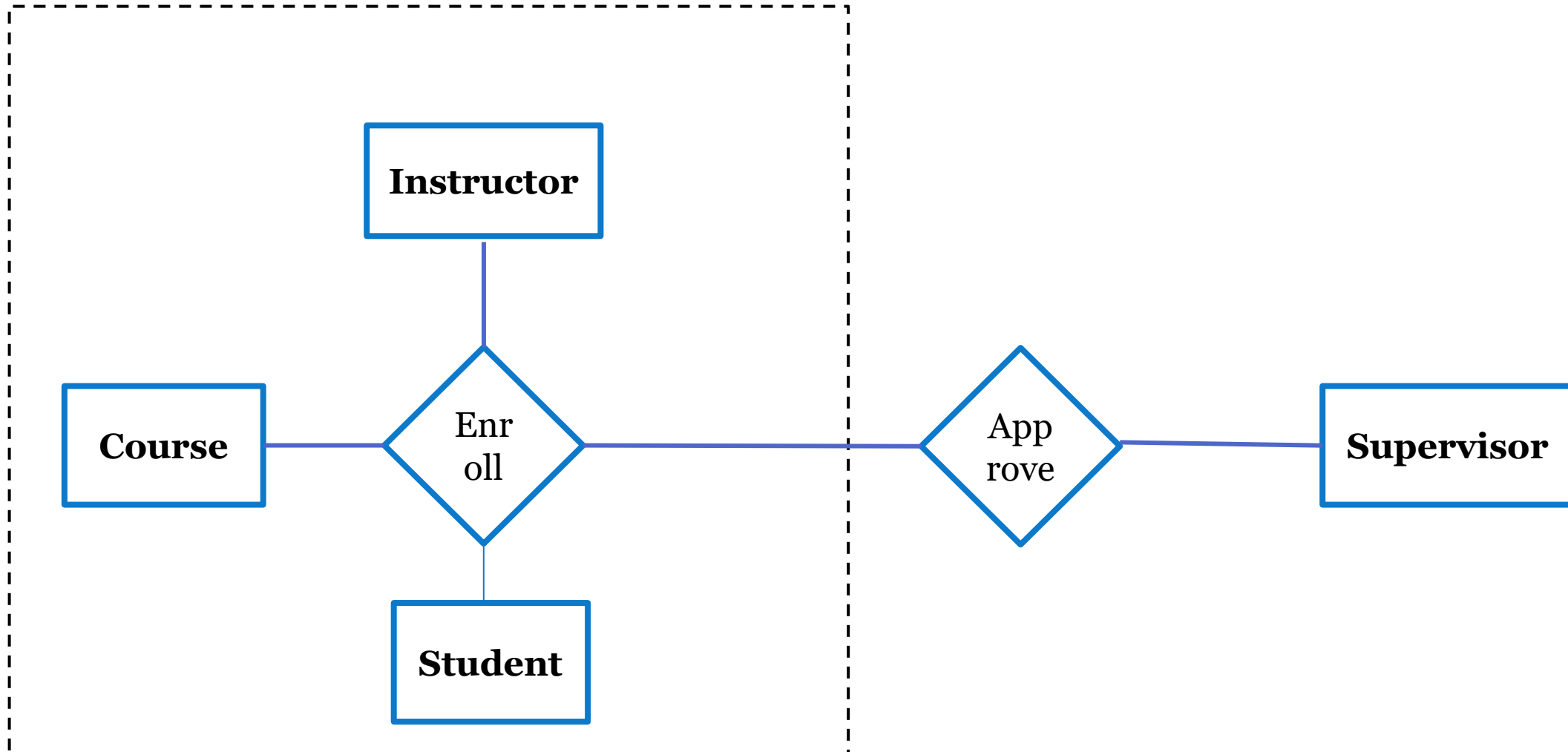
- An “IS-A-PART-OF” (or “Has-A”) relationship represents a **whole–part structure**, where one entity (the whole) is composed of other entities (its parts). Each part contributes to the definition of the whole.
- **Example:**
 - A **Vehicle** consists of an **Engine** and a **Chassis**.
- **Key Points:**
- Both the **whole** and the **parts** have their **own identifiers and attributes**.
- There is **no strong existence dependency**
 - each part can exist independently.
- The relationship mainly shows **composition** or **structural inclusion**, not ownership

Aggregation (Relationship with a Relationship)

- Aggregation is an abstraction technique where a **relationship** itself is treated as an **entity type**, so that it can participate in another relationship. This is mainly used when the relationship **has attributes** or when we need to define **a relationship on a relationship**.



Example: Aggregation



End of Chapter

